

同濟大學

TONGJI UNIVERSITY

《数据采集与集成技术》

实验报告

实验名称	基于大模型的智能代码仓库发现与推荐系统
实验成员	() 吉镓熠 (2352976)
日期	2026 年 1 月 10 日

目 录

1、项目背景	2
2、需求分析	2
2.1 多源异构数据采集需求	2
2.2 数据集成与智能化处理需求	2
2.3 复杂关系挖掘与知识图谱需求	2
2.4 交互式可视化与应用需求	3
3、系统架构与整体设计	3
3.1 整体架构设计	3
3.2 技术栈	4
3.3 项目目录结构	4
3.4 数据库设计	5
4、模块实现原理与详细设计	7
4.1 数据采集模块	7
4.2 数据集成与存储模块	8
4.3 AI 智能分析模块	8
4.4 RAG 向量检索模块	10
4.5 知识图谱构建模块	11
4.6 前端可视化模块	11
5、数据流分析	12
5.1 基础热榜数据流	12
5.2 AI 智能化增强数据流	12
5.3 RAG 向量检索数据流	12
5.4 交互式搜库数据流	13
6、系统实现	13
6.1 数据大屏界面	14
6.2 AI 深度分析界面	14
6.3 贡献者知识图谱界面	15
6.4 AI 智能搜库界面	16
6.5 作者深度画像界面	16
6.6 Chat with Repo 界面	16
7、测试数据	17
7.1 基础采集数据	18
7.2 AI 增值数据	18
7.3 知识图谱关系数据	18
8、数据采集与集成的国内外现状认识	19
9、心得体会	19
9.1 课程收获	19
9.2 存在的不足	20
9.3 改进方向	20
参考文献	20
附录	20

1、项目背景

随着软件技术的飞速发展，开源软件已成为推动技术创新的核心力量，GitHub 作为全球最大的开源代码托管平台，每天涌现出海量的优质项目。然而，面对指数级增长的数据，“信息过载”已成为开发者和技术人员面临的主要挑战。传统的项目筛选方式通常仅依赖 Star 数、Fork 数等量化指标，这往往难以全面反映项目的代码质量与实际功能特点；同时，开发者在寻找特定技术栈解决方案时，需要耗费大量精力手动阅读冗长的 README 文档和源码，筛选效率极低。

在数据采集与集成领域，传统的 ETL（抽取、转换、加载）模式正逐渐向智能化方向演进。随着大语言模型（LLM）技术的爆发，特别是 DeepSeek 等高性能模型的出现，使得计算机具备了对非结构化数据（如技术文档、代码日志）进行深度语义理解的能力。这为我们将零散的互联网数据转化为高价值的结构化情报提供了新的技术路径——即不仅要采得到数据，更要利用 AI 读得懂数据。

基于上述背景，本项目结合个人兴趣与专业所学，设计并实现了一个“GitHub 智能情报中心”。该系统综合运用了 Python 网络爬虫技术、MySQL 数据库集成技术以及 DeepSeek 大模型。项目旨在打破传统数据采集的局限，不仅实现了对 GitHub 热榜（Trending List）、贡献者信息及项目文档的自动化采集与存储，更创新性地引入了 AI 自动评分以及基于 RAG（检索增强生成）的文档智能问答功能，此外，项目还实现了基于全部 Github 仓库的技术栈提取和用户画像生成。该系统能够帮助用户快速捕捉技术热点，显著降低开源项目的筛选与学习成本，是数据采集技术在 AI 时代的一次创新实践。

2、需求分析

本项目的核心需求旨在解决开源情报获取过程中的“数据分散”与“筛选低效”问题，构建一个集采集、存储、分析、展示于一体的智能化系统。系统需求主要划分为以下四个维度：

2.1 多源异构数据采集需求

系统首先需要具备从 GitHub 平台高效获取数据的能力。这包括两类数据源：一是半结构化的网页数据，系统需能够自动访问 GitHub Trending（热榜）页面，解析 HTML 结构，提取项目名称、描述、语言、Stars 数量及增长趋势等关键指标；二是非结构化的文本与 API 数据，系统需深入项目详情页抓取 README 文档（Markdown 格式），并通过 GitHub REST API 获取贡献者列表、用户个人资料及仓库元数据。在采集过程中，系统必须具备应对反爬虫机制的能力，通过模拟 User-Agent、设置随机延时及异常重试机制，确保数据获取的完整性与稳定性。

2.2 数据集成与智能化处理需求

采集到的原始数据通常是杂乱且孤立的，系统需要进行有效的数据清洗与集成。

首先是对数据进行清洗与存储：需去除文本中的多余空格、HTML 标签及特殊符号，将清洗后的数据持久化存储至 MySQL 关系型数据库中，建立项目表、用户表及关联表，确保数据的一致性。

系统对 AI 深度集成，这是系统的核心增值需求。系统调用 DeepSeek 大模型 API，对存储的 README 文档进行语义分析，自动生成中文摘要、提取核心技术栈标签，并依据文档质量和项目活跃度生成推荐评分。

为了实现文档对话功能，系统还需要将长文本进行切片并向量化（Embedding），构建本地向量索引（ChromaDB），为后续的 RAG（检索增强生成）提供数据基础。

2.3 复杂关系挖掘与知识图谱需求

单纯的列表数据无法体现开源社区的协作网络。系统需要挖掘项目与贡献者之间的多对多关系。通过分析仓库的 Contributors 数据，构建数据关联模型，并以知识图谱（Knowledge Graph）

的形式进行可视化。这要求后端能够处理复杂的关联查询，前端能够渲染动态的力导向图，直观展示核心开发者与多个高星项目之间的协作链路。

2.4 交互式可视化与应用需求

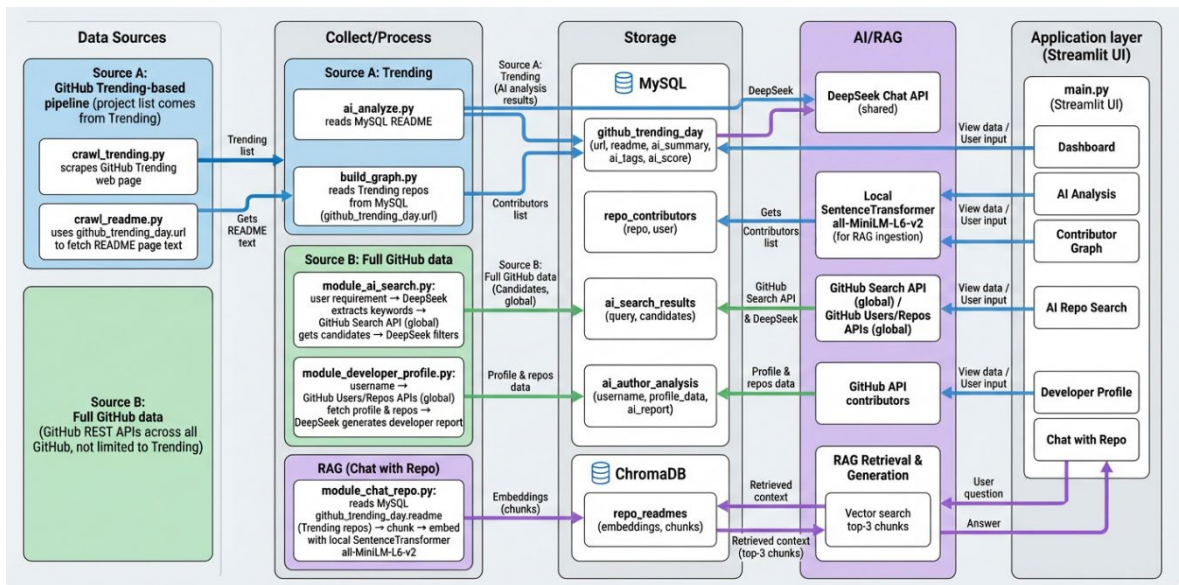
用户界面需基于 Streamlit 框架构建，提供直观的数据交互体验，主要功能如下：

- **数据大屏：** 能够通过图表（如饼图、柱状图）宏观展示今日热榜（GitHub Trending）的语言分布与 Star 数对比。
- **AI 搜索：** 不同于传统的关键词匹配，系统需具备自然语言理解能力，将用户的口语化需求（如“找一个 Python 写的量化框架”）转化为精准的搜索条件，并利用 AI 对搜索结果进行二次筛选与入库。搜索范围是**全部 Github 网站数据**。
- **开发者画像：** 能够根据 GitHub ID 自动生成包含技术栈评估、代码风格分析及代表作点评的深度能力画像。搜索范围是**全部 Github 网站数据**。
- **智能文档问答：** 提供“Chat with Repo”功能，允许用户针对**特定项目(GitHub Trending)**提问，系统利用 RAG 技术基于项目文档事实进行精准回答，辅助用户快速上手代码库。

3、系统架构与整体设计

3.1 整体架构设计

本系统采用模块化的分层架构设计，实现了数据从采集、存储、智能处理到前端展示的完整闭环。系统自下而上分为数据采集层、数据存储层、核心业务层和应用表现层。



● 数据采集层（Data Acquisition Layer）

负责与外部数据源交互。主要包含两类采集模块：一是基于 Requests 和 BeautifulSoup 的网页爬虫，用于抓取 GitHub Trending 榜单及 HTML 页面信息；二是基于 GitHub REST API 的接口采集器，用于获取标准化的用户画像、仓库元数据及贡献者列表。同时包含反爬虫中间件，负责 User-Agent 伪装和请求延时控制。

● 数据存储层（Data Storage Layer）

作为系统的数据中台。采用 MySQL 关系型数据库存储结构化数据（如项目排名、Star 数、清洗后的 README 文本、AI 分析报告）；同时引入 ChromaDB（本地文件形式）作为向量数据库，存储 README 文档的 Embedding 向量数据，为 RAG 检索提供支持。

- 智能处理与业务层（Intelligent Processing Layer）

集成引擎负责数据的清洗、去重以及结构化入库。AI 引擎集成 DeepSeek V3 API，负责文本摘要生成、情感打分、关键词提取及自然语言意图理解。RAG 引擎利用 SentenceTransformer 模型对文档进行向量化，并实现基于语义的上下文检索。

- 应用表现层（Presentation Layer）

基于 Streamlit 框架构建 Web 用户界面。提供数据大屏可视化（ECharts）、AI 智能搜库交互、知识图谱渲染以及 ChatBot 对话窗口，为用户提供直观的操作体验。

3.2 技术栈

本系统的开发环境是 Visual Studio Code 和 MySQL Workbench，Web 框架使用 Streamlit，数据库选用 MySQL 8.0 和 ChromaDB，LLM 选用 DeepSeek V3。

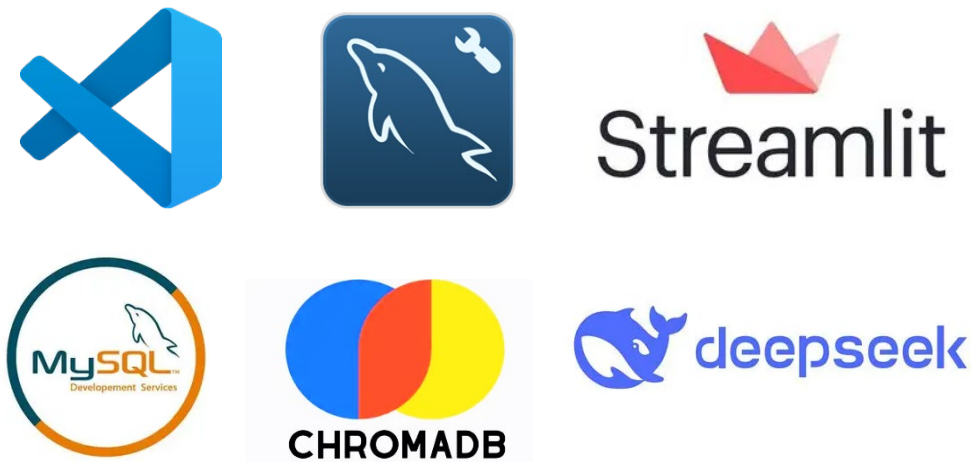


图 2 系统技术栈

本系统的数据采集工具为 Requests、BeautifulSoup4 和 GitHub REST API，在开始数据集成实验前，首先需要准备好实验环境。这个实验依赖的第三方库如下。

表 1 系统依赖的第三方库

类别	库名称	主要功能描述
Web 框架	Streamlit	构建交互式数据大屏与对话界面
可视化	Streamlit-ECharts	渲染知识图谱与动态统计图表
网络请求	Requests	网页抓取、GitHub API 调用及 DeepSeek 交互
数据解析	BeautifulSoup4	解析 HTML DOM 结构提取关键信息
数据库	PyMySQL	MySQL 数据库连接与 CRUD 操作
向量检索	ChromaDB	存储文本向量索引，支持 RAG 语义检索
NLP 模型	Sentence-Transformers	将文本转化为向量（Embedding）
数据清洗	Emoji	清洗文本中的表情符号，确保入库安全

3.3 项目目录结构

本项目采用扁平化的目录结构，所有核心脚本与模块均位于根目录下，通过模块导入实现功能解耦。

/ (项目根目录)	
└─ 数据采集模块	
└─ crawl_trending.py	# 爬虫脚本: 抓取 GitHub Trending 热榜数据 (HTML 解析)
└─ crawl_readme.py	# 爬虫脚本: 抓取项目详情页的 README 文档
└─ build_graph.py	# API 采集脚本: 抓取贡献者数据并构建知识图谱关系
└─ AI 与数据处理模块	
└─ ai_analyze.py	# 批处理脚本: 调用 DeepSeek API 对已采集项目进行摘要和评分
└─ module_ai_search.py	# 功能模块: 实现 “AI 智能搜库” 逻辑
└─ module_chat_repo.py	# 功能模块: 实现 RAG 向量化与问答
└─ module_developer_profile.py	# 功能模块: 实现 “作者画像” 生成
└─ 应用主程序	
└─ main.py	# Streamlit 主程序: Web 界面入口, 集成所有可视化页面
└─ 配置与依赖	
└─ create.sql	# 数据库建表脚本
└─ requirements.txt	# 依赖列表: 列出了项目所需的 Python 第三方库
└─ run.bat	# 批处理脚本: 用于一键启动爬虫和 Streamlit 服务
(生成的本地数据/缓存文件)	
└─ chroma_db/	# (文件夹) ChromaDB 向量数据库的持久化存储目录
└─ gitTrend_db	# (逻辑存在) MySQL 数据库连接目标

3.4 数据库设计

本系统选用 MySQL 8.0 作为核心关系型数据库, 数据库命名为 `gitTrend_db`。为了确保能够完整存储 GitHub 项目描述及 README 文档中广泛存在的 Emoji 表情符号及多语言字符, 数据库字符集统一采用 `utf8mb4`, 排序规则采用 `utf8mb4_unicode_ci`。

系统主要包含四张核心数据表, 分别承载项目详情数据与贡献者关系数据。为了支持系统的 “AI 智能搜库” 和 “作者深度画像” 功能, 设计了两张业务表, 用于持久化存储用户的交互历史及 AI 生成的高价值分析报告, 实现了 “一次生成, 永久调用” 的缓存机制。

表 2 项目热榜主表 (`github_trending_day`)

类别	数据类型	属性	说明	设计意图
id	INT	PK, Auto Inc	主键 ID	唯一标识每一条记录
ranking	INT	Not Null	榜单排名	保留原始爬取时的热度顺序
repo	VARCHAR (255)	Not Null	仓库标识	格式为 “owner/repo”, 作为项目唯一索引
lang	VARCHAR (50)	Nullable	编程语言	用于后续的数据大屏饼图统计
stars	VARCHAR (50)	Nullable	总 Star 数	存储原始展示数据 (如 1.2k), 清洗时转换为数值

续表 2

类别	数据类型	属性	说明	设计意图
description	TEXT	Nullable	项目简介	存储清洗了 HTML 标签后的文本
readme	LONGTEXT	Nullable	README 文档	关键设计：使用 LONGTEXT 存储长篇 Markdown 文档，为 RAG 检索提供语料
ai_summary	VARCHAR (500)	Nullable	AI 摘要	存储 DeepSeek 生成的中文短摘要，解决信息过载
ai_score	INT	Nullable	AI 推荐分	存储 1-10 的量化评分，便于用户筛选
time	DATETIME	Default Now	爬取时间	记录数据版本，便于分析热度趋势

表 3 贡献者关系表 (repo_contributors)

类别	数据类型	属性	说明	设计意图
id	INT	PK, Auto Inc	主键 ID	唯一标识
repo_url	VARCHAR (255)	Not Null	关联仓库	逻辑外键，关联主表中的项目
contributor_name	VARCHAR (100)	Not Null	贡献者用户名	图谱中的“用户节点”名称
avatar_url	VARCHAR (255)	Nullable	头像链接	用于前端 ECharts 图谱中渲染用户节点图片
crawl_time	DATETIME	Default Now	爬取时间	数据时效性标记

表 4 AI 智能搜库结果表 (ai_search_results)

类别	数据类型	属性	说明	设计意图
id	INT	PK	主键 ID	唯一标识
search_prompt	TEXT	Not Null	原始需求	记录用户输入的自然语言（如“找个 Vue 后台”），用于后续分析用户行为
keyword	VARCHAR (255)	Nullable	提取关键词	存储 AI 从长文本中提取出的核心搜索词（如“Vue3 Admin”）
repo	VARCHAR (255)	Not Null	仓库名称	搜索结果的项目标识
ai_reason	VARCHAR (500)	Comment	AI 推荐理由	存储 AI 根据需求生成的个性化推荐语，体现智能集成的价值

续表 4

类别	数据类型	属性	说明	设计意图
match_score	INT	Comment	匹配得分	AI 计算的项目与用户需求契合度（0-100），用于结果排序
created_at	DATETIME	Default Now	创建时间	记录搜索发生的时间

表 5 贡献者关系表（repo_contributors）

类别	数据类型	属性	说明	设计意图
id	INT	PK	主键 ID	唯一标识
username	VARCHAR (100)	UNIQUE	用户名	唯一索引：确保每个开发者在库中只有一份最新的画像，防止数据冗余
bio	TEXT	Nullable	用户简介	GitHub 原始简介
ai_report	TEXT	Comment	AI 分析报告	存储 DeepSeek 生成的 Markdown 格式技术评估报告
tech_stack	VARCHAR (255)	Comment	技术栈	提取出的核心技术标签（如 Python, React），便于快速检索
updated_at	DATETIME	On Update	更新时间	缓存策略：利用 MySQL 的自动更新特性，当画像重新生成时自动刷新时间

4、模块实现原理与详细设计

本系统采用模块化设计思想，主要包含数据采集、数据集成与存储、AI 智能分析、RAG 向量检索、知识图谱构建及前端可视化六大核心模块。

4.1 数据采集模块

对应文件：crawl_trending.py、crawl_readme.py、module_developer_profile.py。

本模块作为整个系统的源头与基石，负责从互联网获取多源异构数据。针对 GitHub 平台数据形态的多样性，模块设计了“网页爬虫”与“API 接口采集”并行的双重策略，实现了对半结构化网页数据与非结构化 JSON 数据的全面覆盖，为后续的存储与分析提供了丰富的数据原料。

● 动态网页与文档爬取

针对 GitHub Trending（热榜）及项目详情页，模块主要采用 HTML 解析技术（对应 crawl_trending.py 和 crawl_readme.py）。系统利用 requests 库模拟浏览器发起 HTTP GET 请求，获取页面的静态 HTML 源码。随后，引入 BeautifulSoup4 解析器构建 DOM 树，通过精确的 CSS 选择器（如 article.Box-row）定位列表中的每一个项目容器。系统能够自动遍历这些节点，清洗并提取出 Repository Name（仓库名）、Description（简介）、Stars（关注数）、Forks（分支数）等关键元数据。此外，为了获取项目的深度信息，模块还进一步深入详情页，抓取<article class="markdown-body">标签下的 README.md 原始内容，并进行初步的格式清洗，作为后续 RAG 检索的语料库。

● 高性能 API 数据采集

针对结构化程度较高的用户画像和贡献者网络数据，模块封装了标准的 GitHub REST API 调用逻辑（对应 `module_developer_profile.py`）。相较于网页爬虫，API 采集能够提供更精准的字段（如用户 Bio、头像 URL、详细贡献记录）。为了突破 GitHub 对匿名请求的严格频率限制（Rate Limit），模块在 HTTP 请求头（Header）中动态注入了 `Authorization: token <GITHUB_TOKEN>` 鉴权凭证，显著提升了每小时的请求配额。同时，模块内置了健壮的异常处理机制，通过判断 HTTP 状态码来自动识别 403 Forbidden（触发限流）和 404 Not Found（资源不存在）等异常情况，并执行相应的跳过或重试逻辑，确保程序的稳定性。

● 反爬虫与访问控制策略

为了应对目标网站的反爬虫机制并减轻服务器负载压力，本模块实施了拟人化的访问控制策略。在网络请求层，模块强制设置了包含常用浏览器特征的 `User-Agent`，防止被识别为自动化脚本。更关键的是，模块引入了随机延时机制，在连续请求之间插入符合正态分布的随机等待时间（调用 `random.uniform(1.5, 3.5)` 秒）。这种策略有效模拟了人类用户阅读页面时的自然停顿，极大降低了因高频访问导致 IP 被封禁的风险，保障了数据采集任务的可持续性。

4.2 数据集成与存储模块

对应文件：`create.sql` 和 `pymysql` 相关操作（建表语句）

本模块作为连接采集层与应用层的中间件，承担着数据清洗（Cleaning）、转换（Transformation）与持久化存储（Loading）的关键职责。针对项目数据的多维特性，模块设计了关系型存储与向量型存储相结合的混合架构，确保了数据的高可用性与语义完整性。

● 数据清洗与预处理

采集到的原始数据往往包含大量噪声，直接入库会影响后续分析的准确性。因此，在数据持久化之前，模块首先执行严格的 ETL 流程。

第一步利用 Python 的 `re` 正则表达式模块对长文本进行清洗，剔除冗余的 HTML 标签、脚本代码以及连续的空行与制表符，将非结构化的网页文本转化为纯净的纯文本格式。随后，针对 GitHub 社区中广泛使用的 Emoji 表情符号（如🔥、🚀），模块引入 `emoji.demojize` 方法将其转换为 ASCII 文本描述（如`:fire:`、`:rocket:`）。这一处理不仅保留了情感信息，有效防止了因特殊字符编码冲突导致的数据库写入异常，增强了系统的鲁棒性。

● 关系型数据持久化

对于具有明确 Schema 的结构化数据（如项目元数据、排名、统计指标），系统选用 MySQL 8.0 作为核心存储引擎。设计了核心数据表 `github_trending_day`，字段涵盖了 `ranking`（当前热度排名）、`repo`（项目唯一标识符）、`stars`（量化热度）以及 `readme`（项目文档）。

考虑到开源项目的国际化特征，数据库连接与表结构均强制指定为 `utf8mb4` 字符集。相较于传统的 `utf8`，`utf8mb4` 能够完整支持 4 字节的 Unicode 字符（包括生僻字与复杂表情），确保了全球多语言项目信息的无损存储。

● 向量化存储与语义索引

为了挖掘非结构化 README 文档的深层价值并支持 AI 语义检索，本模块引入了 ChromaDB 向量数据库。系统将清洗后的 README 长文本输入 Embedding 模型，转化为高维稠密向量（Vector）。这些向量蕴含了文本的语义特征，被存储于本地 ChromaDB 实例中。不同于 MySQL 的关键词匹配，向量存储允许系统在几何空间中计算文本的相似度，从而为“Chat with Repo”功能中的 RAG（检索增强生成）机制提供了底层的索引支持，实现了从文本存储到语义存储的跨越。

4.3 AI 智能分析模块

对应文件：`ai_analyze.py` 和 `module_ai_search.py`。

本模块负责对采集到的多源异构数据进行深度语义理解与结构化转换。传统的关键词匹配无法理解项目的真实用途，而本模块利用 DeepSeek V3 大语言模型，将非结构化的文本数据（如 Markdown 格式的 README、用户的自然语言查询）转化为结构化的情报（摘要、评分、技术栈标签、搜索意图），实现了从数据采集到知识提取的质的飞跃。本模块主要实现以下功能：

- **智能摘要与评分:** 自动阅读项目长文档, 提取核心功能, 并根据文档完整度与趣味性输出 1-10 的推荐分。
- **技术栈实体抽取:** 从文本中识别并标准化编程语言与框架实体 (如将 “Vue.js 3” 统一提取为 “Vue3”)。
- **自然语言意图理解:** 在搜索场景下, 将用户模糊的口语化需求 (如 “找个 Python 爬虫”) 转化为精准的搜索关键词和筛选条件。

本模块的核心在于利用生成式预训练模型 (GPT 架构) 进行文本推断。其背后的数学原理主要包含自回归生成和温度采样两个方面。

● 自回归生成模型

DeepSeek 模型本质上是一个概率预测模型。给定输入的项目文档序列 $X = (x_1, x_2, \dots, x_m)$ (即 Prompt), 模型的目标是生成目标序列 $Y = (y_1, y_2, \dots, y_n)$ (即 JSON 格式的分析结果)。

根据概率链式法则, 整个序列生成的联合概率被分解为每一步条件概率的乘积:

$$P(Y|X) = \prod_{t=1}^n P(y_t | y_1, y_2, \dots, y_{t-1}, X; \theta)$$

其中, y_t 是第 t 步生成的 Token (词元), θ 代表模型训练好的亿级参数。模块通过最大化该概率来生成最合理的分析结果。

● 温度采样机制

在代码 `ai_analyze.py` 中, 我们设置了 `temperature=0.3`。这是为了控制输出的随机性与确定性。

模型输出层使用 Softmax 函数将逻辑值 (Logits) 转化为概率分布。引入温度参数 T 后, 第 i 个词元的被选概率 p_i 计算公式为:

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

z_i 是模型输出的原始得分。当 $T \rightarrow 0$ 时 (如本项目设置的 0.3), 概率分布变得尖锐, 模型倾向于选择置信度最高的词, 从而保证提取出的技术栈和评分具有高度的准确性和稳定性, 避免 AI 产生幻觉。

为了确保输出结果能直接被程序解析并存入 MySQL 数据库, 本模块采用了结构化提示工程 (Structured Prompt Engineering) 技术。

● 系统级指令

我们将 AI 的角色设定为开源项目技术分析师, 并强制约束输出格式为 JSON。

Prompt 示例:

```
system_prompt = """
你是一个开源项目技术分析师。请根据用户提供的 GitHub README 内容, 进行以下分析:
    1. summary: 用中文一句话概括这个项目是做什么的 (不超过 50 字)。
    2. tags: 提取项目涉及的核心编程语言或技术栈 (不超过 5 个, 英文逗号分隔)。
    3. score: 根据文档完整度和项目有趣程度, 给出一个推荐分数 (1-10 的整数)。

请务必只返回标准的 JSON 格式, 不要包含 Markdown 标记 (如 ``json)。
格式示例:
{"summary": "一个基于 Python 的爬虫框架", "tags": "Python, Web, Crawler",
 "score": 8}
"""
```

● JSON 模式强制

在调用 API 时, 我们在 Payload 中显式指定了响应格式, 确保返回的数据严格符合语法要求, 避免了传统 NLP 任务中复杂的正则提取步骤:

```
payload = {
    "model": "deepseek-chat",
    "messages": [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": f"README 内容如下:\n{content_snippet}"},
    ],
    "temperature": 0.3, # 温度低一点，输出更稳定
    "response_format": { "type": "json_object" }
}
```

4.4 RAG 向量检索模块

对应文件：module_chat_repo.py。

本模块实现了系统的核心创新功能——与项目文档对话。传统的 LLM（大语言模型）存在知识截止和幻觉问题，无法准确回答关于特定 GitHub 项目（特别是新发布项目）的细节问题。本模块基于检索增强生成（Retrieval-Augmented Generation, RAG）架构，通过引入外部知识库（即采集到的 README 文档），让 AI 能够基于事实进行精准回答，实现了“外挂知识库”的智能问答。其工作流程分为离线索引和在线检索两个阶段：

- 阶段一：数据向量化与索引（Indexing）

- 文本切片（Text Chunking）：为了适应 LLM 的上下文窗口限制，系统采用滑动窗口算法将长篇 README 切分为多个短片段。设置 chunk_size=500 字符，overlap=50 字符，重叠切片的设计保证了语义的连贯性，避免上下文在切分处丢失。
- 向量嵌入（Embedding）：使用本地部署的 SentenceTransformer 模型（all-MiniLM-L6-v2），将文本片段映射为高维稠密向量。
- 持久化存储：将向量及对应的元数据存入 ChromaDB 本地向量数据库中。

- 阶段二：语义检索与生成（Retrieval & Generation）

- 向量化：将用户的自然语言提问（如“这个库怎么安装？”）转化为同维度的查询向量。
- 近邻搜索（ANN Search）：在 ChromaDB 中检索与查询向量距离最近的 Top-K 个文档片段。
- 提示词组装：将检索到的片段作为“参考上下文（Context）”，结合用户问题组装成最终的 Prompt，发送给 DeepSeek 模型生成答案。

本模块的核心在于利用向量空间模型来衡量语义的相似度。

- 向量嵌入

定义嵌入函数 f_θ （即 SentenceTransformer 模型），它将任意长度的文本 T 映射到 d 维实数向量空间 $\mathbb{R}^d$ 中。对于第 i 个文档片段 doc_i 和用户查询 $query$ ，其向量表示为：

$$\vec{v}_{doc_i} = f_\theta(doc_i), \quad \vec{v}_{query} = f_\theta(query)$$

在本系统中，模型输出维度 $d=384$ 。在此空间中，语义相似的文本在几何距离上更接近。

- 余弦相似度

为了找到与用户问题最相关的文档，系统计算查询向量与文档向量之间的夹角余弦值。这是衡量高维空间中向量方向一致性的标准指标。公式如下：

$$\text{Score}(query, doc_i) = \cos(\theta) = \frac{\vec{v}_{query} \cdot \vec{v}_{doc_i}}{|\vec{v}_{query}| |\vec{v}_{doc_i}|} = \frac{\sum_{j=1}^d v_{query}^{(j)} v_{doc_i}^{(j)}}{\sqrt{\sum_{j=1}^d (v_{query}^{(j)})^2} \sqrt{\sum_{j=1}^d (v_{doc_i}^{(j)})^2}}$$

其中， $\cdot$ 表示点积， $|\cdot|$ 表示向量的模。Score 值越接近 1，表示两者语义越相关。

- RAG 生成概率

在传统的生成模型中，答案 Y 仅依赖于输入 X 。而在 RAG 架构中，答案生成概率依赖于检索到的相关文档集合 $Z = \{z_1, z_2, \dots, z_k\}$ 。模型的目标是最大化以下概率：

$$P(Y|X) \approx \sum_{z \in Z} P(Y|X, z)P(z|X)$$

在本系统的实现中，我们将 Top-K 文档直接拼接至 Prompt 中，简化为条件生成任务： $P(Y|X, \text{Concat}(z_1 \dots z_k))$ ，从而强制模型依据检索到的事实进行回答。

具体来说，我们选用 all-MiniLM-L6-v2 模型，该模型在 HuggingFace 排行榜上拥有极高的性能/体积比，适合单机环境运行，无需依赖外部 API 计算向量，保护了数据隐私。同时，我们选用 ChromaDB，它是一个开源的原生向量数据库，支持持久化存储（PersistentClient），确保重启系统后索引数据不丢失。在构建 Prompt 时，使用了“如果参考文档中没有相关信息，请诚实地说...”的指令，有效抑制了模型的幻觉现象，提高了系统的可靠性。

4.5 知识图谱构建模块

对应文件：build_graph.py。

本模块致力于挖掘开源社区中隐藏的社交协作网络，将离散的项目与开发者数据转化为可视化的图谱结构。传统的列表视图只能展示单一维度的信息，而本模块通过构建异构信息网络（Heterogeneous Information Network），直观揭示了“谁在维护核心项目”以及“哪些项目拥有共同的贡献者”。

● 图论建模

模块基于图论思想对采集到的贡献者数据进行数学建模。我们将整个开源协作网络抽象为一个图 $G = (V, E)$ 。

➤ **节点集合 V** ：包含两种类型的节点。一类是项目节点 V_{repo} （可视化为绿色大圆点），代表 GitHub 上的开源仓库；另一类是用户节点 V_{user} （可视化为蓝色小圆点），代表核心贡献者。

➤ **边集合 E** ：代表“贡献（Contribute）”这一实体关系。当且仅当用户 u 出现在项目 r 的贡献者列表中时，图谱中存在一条无向边 $e_{u,r} \in E$ 。这种多对多的连接结构能够有效识别出跨项目活跃的开发者的。

● 力导向布局算法

为了在二维平面上美观且清晰地展示复杂的网络结构，本模块采用了 Force-Directed Algorithms（力导向布局算法）进行渲染。该算法将图谱模拟为一个物理系统：

➤ **库仑斥力**：任意两个节点之间存在互斥力，模拟电荷排斥，防止节点重叠，确保图谱布局的舒展性。斥力公式为 $F_{rep}(d) = -\frac{k^2}{d}$ 。

➤ **胡克引力**：仅在存在边相连的节点之间存在引力，模拟弹簧拉伸，使相关的项目与作者相互靠近，形成聚类。引力公式为 $F_{att}(d) = \frac{d^2}{k}$ 。

其中 d 为节点间距离， k 为理想边长。系统通过迭代计算使系统的总能量趋向最小，最终达到动态平衡状态，自然呈现出社区的聚类特征。

4.6 前端可视化模块

对应文件：main.py。

本模块是用户与系统交互的窗口，基于 Streamlit 框架构建了一个响应式的单页面应用（Single Page Application, SPA）。设计遵循“高内聚、低耦合”原则，将后端复杂的 AI 推理与数据检索逻辑封装在简洁的 Web 界面之后，为用户提供流畅的数据洞察体验。

前端界面放弃了传统的 HTML/CSS 硬编码模式，转而采用 Streamlit 的组件化开发模式。项目利用 st.sidebar 构建全局侧边导航栏，实现六大功能模块的无缝切换；使用 st.columns 和 st.container 实现栅格化网格布局，使得数据大屏中的 KPI 指标、柱状图与饼图能够自适应排列，提升视觉层次感。

针对 Streamlit 每次交互重新运行脚本的特性，模块利用 `st.session_state` 构建了持久化的会话缓存机制。这对于“Chat with Repo”功能至关重要，它确保了用户在与 AI 多轮对话时，历史聊天记录不会因页面刷新而丢失，保证了上下文的连续性。

为了突破 Streamlit 原生图表的局限性，模块集成了 `streamlit-echarts` 库，实现了 Python 后端与 JavaScript 前端的桥接。后端 Python 将处理好的 Pandas DataFrame 或 NetworkX 图数据转换为标准的 JSON 格式对象（ECharts Option）。前端接收 JSON 配置后，在 HTML Canvas 上绘制高性能的动态图表。这使得知识图谱支持缩放、拖拽和鼠标悬停（Hover）显示详细信息等高级交互，极大地增强了数据的可探索性。

5、数据流分析

本系统的数据流转遵循“采集（Acquisition）→存储（Storage）→增强（Enrichment）→应用（Application）”的生命周期。根据业务逻辑，系统主要包含以下四条核心数据链路：

5.1 基础热榜数据流

这是系统的基础数据链路，负责将互联网上的半结构化网页转化为数据库中的结构化记录。

表 6 基础热榜数据流

数据源（Source）	GitHub Trending 页面（HTML）。
抽取（Extract）	<code>crawl_trending.py</code> 脚本发起 HTTP 请求，获取原始 HTML。
转换（Transform）	利用 BeautifulSoup 解析 DOM 树，清洗字段（去除换行符、提取数字），处理 Emoji 表情。
加载（Load）	通过 pymysql 执行 INSERT 语句，将清洗后的数据存入 MySQL 的 <code>github_trending_day</code> 表。

5.2 AI 智能化增强数据流

这是体现“数据集成”价值的核心链路。原始数据经过 AI 处理后，产生了更高价值的衍生数据。

表 7 AI 智能化增强数据流

输入（Input）	从 MySQL 中读取未处理的 README 长文本字段。
处理（Process）	<code>ai_analyze.py</code> 将文本截断并封装进 System Prompt，调用 DeepSeek API。模型进行语义推断，输出 JSON 格式的摘要、评分和标签。
回写（Update）	解析返回的 JSON 数据，执行 UPDATE 语句将 <code>ai_summary</code> 、 <code>ai_score</code> 等字段回写至 MySQL 对应记录中，完成数据的原地增值。

5.3 RAG 向量检索数据流

该链路负责将结构化文本转化为机器可理解的数学向量，服务于“Chat with Repo”功能。

表 8 RAG 向量检索数据流

输入（Input）	从 MySQL 中的 README 文本数据。
切片（Chunking）	module_chat_repo.py 将长文本按 500 字符为窗口进行滑动切分。
向量化（Embedding）	调用 SentenceTransformer 模型，将文本片段映射为 384 维的高维向量。
索引存储（Indexing）	将向量值（Vector）与元数据（Metadata）存入 ChromaDB 本地向量库，构建 ANN（近似最近邻）索引。
检索交互（Retrieval）	用户提问→转化为查询向量→ChromaDB 匹配 Top-K 片段→注入 DeepSeek 上下文→生成最终回答。

5.4 交互式搜库数据流

该链路处理用户的实时查询需求，体现了用户意图到数据落地的过程。

表 9 交互式搜库数据流

意图识别	用户输入自然语言需求→module_ai_search.py 调用 AI 提取核心关键词。
API 聚合	使用提取的关键词调用 GitHub Search API，获取 Top-N 候选列表。
AI 筛选	将候选列表二次发送给 AI，根据原始需求进行相关性打分和理由生成。
持久化	将最终筛选出的高质量项目及其 AI 评价存入 ai_search_results 表，供前端展示和历史回溯。

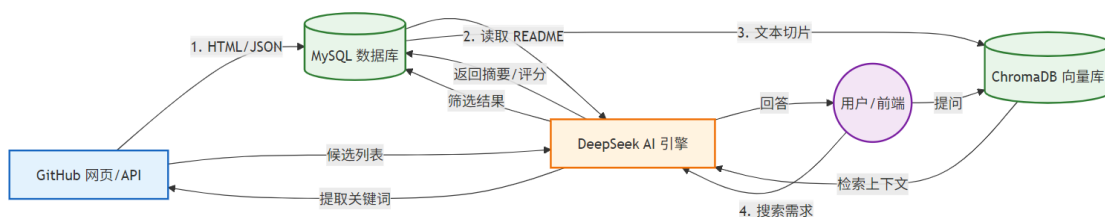


图 3 系统数据流图

6、系统实现

本系统在实现层面严格遵循 ETL（Extract - Transform - Load）数据处理范式，重点围绕“如何采集”与“如何集成”展开技术设计，构建了一条从多源异构数据获取到智能化融合的完整实现链路。在数据采集阶段，针对 GitHub 平台网页结构复杂、数据类型多样的特点，采用了“网页爬虫+官方 API”的双通道采集方案，实现了对榜单信息、项目文档及开发者画像等数据的高效获取。

在网页采集方面，系统基于 DOM 解析机制，通过 requests 与 BeautifulSoup 精确定位页面结构，实现对 Trending 榜单及 README 文档的稳定抓取；同时引入随机延时与增量采集策略，有效降低反爬风险并提升整体采集效率。在结构化数据获取方面，系统直接调用 GitHub REST API，并通过 Token 鉴权突破匿名访问频率限制，结合异常状态码处理机制，保障了高并发场景下的数据采集稳定性。

在数据集成阶段，系统不仅完成了传统意义上的数据清洗与入库，还进一步引入 LLM 能力对数据进行智能增强。通过统一的清洗与规范化流程，将原始文本、结构化指标与 LLM 生成的摘要、标签及评分进行融合存储，实现了客观数据和主观智能分析的有机集成。此外，系统同步构建了向量化索引，将关系型数据库与向量数据库进行双库联动，为后续语义检索与对话式应用提供底层支持。

整体而言，本实现方案不仅清晰回答了“数据从哪里来、如何处理、如何融合”的问题，也体现了从基础采集到智能集成的技术递进过程，为系统功能实现和应用扩展奠定了坚实的数据基础。本系统基于 Streamlit 框架构建了统一的 Web 交互平台，后端集成了数据采集、数据库管理及 LLM 分析模块。系统共包含六大核心功能界面，具体实现逻辑如下：

6.1 数据大屏界面

作为系统的首页，宏观展示 GitHub 今日热榜的统计数据。包含项目总数、最高 Star 数、编程语言分布等关键指标，以及 Top 10 项目的柱状图对比。

数据源从 MySQL 的 github_trending_day 表中读取由 crawl_trending.py 采集并清洗后的结构化数据。数据处理时使用 Pandas 读取 SQL 数据为 DataFrame，对 stars 字段进行类型转换（去除 k 和逗号转为浮点数），并计算语言频率分布。

前端实现时利用 st.metric 组件展示核心指标（KPI）；利用 st.bar_chart 绘制 Star 数排名前十的项目对比图；引入 streamlit_echarts 库，配置 ECharts 的 pie（饼图）参数，动态渲染编程语言的市场占比。



图 3 数据大屏界面截图

6.2 AI 深度分析界面

展示经过 DeepSeek 大模型深度解读后的项目情报。用户可以根据“AI 推荐分”筛选高质量项目，查看由 AI 生成的中文摘要和技术栈标签。

AI 处理依赖 ai_analyze.py 脚本（后台运行），通过 Prompt Engineering 将 README 文本发送

给 DeepSeek API，要求其返回 JSON 格式的评分（1-10 分）与摘要。数据集成时将 API 返回的 ai_summary、ai_tags、ai_score 字段更新至数据库。

前端实现使用 st.slider 实现交互式筛选（例如：只看评分>7 的项目）；使用 st.container 和 st.divider 布局，左侧显示高亮评分，右侧展示摘要与技术栈，解决了传统列表信息密度过低的问题。



图 4 AI 深度分析界面截图

6.3 贡献者知识图谱界面

通过力导向图（Force-Directed Graph）直观展示“项目”与“贡献者”之间的多对多协作关系，帮助用户发现核心开发者。

关系构建时依赖 build_graph.py 脚本，调用 GitHub API 获取每个仓库的 Contributors 列表，并存入 repo_contributors 表。节点与边是在 Pandas 中处理数据，将仓库定义为“绿色节点”，贡献者定义为“蓝色节点”，两者之间的贡献关系定义为边。

前端实现利用 ECharts 的 graph 类型进行渲染。设置 roam=True 允许用户缩放拖拽，设置 layout='force'让图谱自动布局，直观呈现开源社区的社交网络结构。



图 5 贡献者知识图谱界面截图

6.4 AI 智能搜库界面

用户输入自然语言需求（如“找一个 Python 写的量化交易框架”），系统自动提取关键词、搜索 GitHub、并利用 AI 筛选出最匹配的结果存入数据库。

调用 DeepSeek 提取用户输入中的核心关键词（如“Quantitative Trading Python”），使用提取的关键词调用 GitHub Search API 获取 Top-30 候选列表，将候选列表回传给 DeepSeek，根据用户原始需求进行相关性打分，过滤掉不相关的项目，将筛选后的结果存入 ai_search_results 表。

前端实现使用 st.status 组件展示“提取关键词→搜索→筛选→入库”的实时进度条，提升用户体验。搜索完成后，自动渲染结果卡片，展示 AI 生成的推荐理由。



图 6 AI 智能搜库界面截图

6.5 作者深度画像界面

输入 GitHub 用户名，系统自动抓取其个人资料及原创仓库，生成一份包含技术栈、代码风格及代表作分析的深度评估报告。

后端实现调用 module_developer_profile.py：并发调用 GitHub API 获取用户 Profile 和 Repositories 数据。构造包含用户简介和仓库列表的长 Prompt，请求 DeepSeek 生成 Markdown 格式的技术报告。利用 INSERT ... ON DUPLICATE KEY UPDATE 语句，将生成好的画像存入 MySQL，避免重复消耗 Token。

前端实现时左侧展示头像与简介，右侧展示 AI 生成的 Markdown 报告；利用 ECharts 饼图分析该作者所有仓库的语言构成。

6.6 Chat with Repo 界面

基于 RAG（检索增强生成）技术，允许用户与选定项目的 README 文档进行对话，快速了解项目细节（如安装步骤、配置方法）。

后端实现调用 module_chat_repo.py：使用 SentenceTransformer 模型将项目文档切片并转化为向量，存储于 ChromaDB。当用户提问时，系统在 ChromaDB 中检索 Top-3 相关片段。将检索到的片段拼接到 Prompt 中：“请基于以下参考文档回答用户问题...”，发送给 DeepSeek 生成最终答案。

前端实现使用 `st.chat_message` 组件构建类似 ChatGPT 的对话气泡界面；利用 `st.session_state` 维护聊天上下文历史，确保对话的连续性。



图 7 作者深度画像界面截图



图 8 Chat with Repo 界面截图

7、测试数据

本系统成功构建了高可用的数据采集与集成链路，完成了从 GitHub 动态网页到 MySQL 关系型数据库的完整映射。通过自动化爬虫脚本，系统有效抓取了包括项目排名、Stars 数、README 长文档在内的多维数据，并采用 `utf8mb4` 字符集与 Emoji 编码策略，确保了全球化多语言字符及表情符号的无损存储，验证了 ETL 流程的稳定性与数据库设计的健壮性。

在数据集成基础上,系统通过调用DeepSeek大模型实现了对原始非结构化数据的深度加工。测试数据展示了AI自动回写的项目中文摘要、技术栈标签及1-10维度的量化评分,并成功将文本切片向量化存入ChromaDB索引中。同时,系统构建了清晰的“项目-贡献者”关联数据,为前端的智能化搜索、知识图谱渲染及RAG文档对话功能提供了坚实且高价值的数据支撑。

具体测试数据展示如下:

7.1 基础采集数据

展示 ranking、repo、stars 字段,证明爬虫成功抓取了热榜基础信息。

id	ranking	repo	url	description	lang	stars	forks	added_stars
1	1	/thedotmact/claude-mem	https://github.com/thedotmact/claude-mem	A Claude Code plugin that automatically captur...	TypeScript	12,232	767	703 stars today
2	2	/google/googletest	https://github.com/google/googletest	GoogleTest - Google Testing and Mocking Frame...	C++	37,822	10,652	9 stars today
3	3	/Lissy93/web-check	https://github.com/Lissy93/web-check	All-in-one OSINT tool for analysing any website	TypeScript	28,781	2,316	783 stars today
4	4	/microsoft/PowerToys	https://github.com/microsoft/PowerToys	Microsoft PowerToys is a collection of utilities th...	C#	127,727	7,585	330 stars today
5	5	/protocolbuffers/protobuf	https://github.com/protocolbuffers/protobuf	Protocol Buffers - Google's data interchange for...	C++	70,206	15,995	133 stars today
6	6	/ChromeDevTools/chrome-devtools-mcp	https://github.com/ChromeDevTools/chrome-de...	Chrome DevTools for coding agents	TypeScript	19,155	1,186	90 stars today
7	7	/memvid/memvid	https://github.com/memvid/memvid	Memory layer for AI Agents. Replace complex R...	Rust	11,588	966	233 stars today
8	8	/patchy631/ai-engineering-hub	https://github.com/patchy631/ai-engineering-hub	In-depth tutorials on LLMs, RAGs and real-world...	Jupyter Notebook	25,469	4,129	158 stars today
9	9	/DrewThomason/ebook2audiobook	https://github.com/DrewThomason/ebook2aud...	Generate audiobooks from e-books, voice donin...	Python	16,835	1,354	69 stars today
10	10	/marcelscruz/public-apis	https://github.com/marcelscruz/public-apis	A collaborative list of public APIs for developers	JavaScript	7,414	684	454 stars today
11	11	/prateek-chaubey/YTPro	https://github.com/prateek-chaubey/YTPro	Youtube client with older Android version suppo...	JavaScript	1,548	127	261 stars today
12	12	/MiroMindAI/MiroThinker	https://github.com/MiroMindAI/MiroThinker	MiroThinker is a open-source search agent suite...	Python	3,141	200	511 stars today

time	readme	ai_summary	ai_tags	ai_score
2026-01-08 18:38:23	cn 中文 • jp 日本語 • pt Português • ko 한국...	一个为Claude Code设计的持久内存压缩系统...	JavaScript, Bun, SQLite, Chroma, HTTP	8
2026-01-08 18:38:23	GoogleTest Announcements Documentation Up...	Google开发的C++单元测试框架,支持xUnit...	C++, Testing, xUnit, Google, Framework	9
2026-01-08 18:38:23	Web-Check Comprehensive, on-demand open s...	一个提供网站全面开源情报分析的工具,用...	Web, Security, Docker, DNS, SSL	8
2026-01-08 18:38:23	Microsoft PowerToys Microsoft PowerToys is a c...	一个微软开发的Windows系统增强工具集,...	Windows, C++, .NET, PowerShell, WinGet	8
2026-01-08 18:38:23	Protocol Buffers - Google's data interchange for...	Google开发的用于序列化结构化数据的数据语言...	C++, Java, Python, Bazel, Protocol Buffers	9
2026-01-08 18:38:23	Chrome DevTools MCP chrome-devtools-mcp let...	一个让AI编程助手通过MCP协议控制Chrome...	Node.js, Chrome DevTools, Puppeteer, MCP	8
2026-01-08 18:38:23	Memvid is a single-file memory layer for AI agen...	一个为AI代理提供便携式、单文件内存层的...	Rust, Python, Node.js, AI, Memory	9
2026-01-08 18:38:23	AI Engineering Hub Welcome to the AI Engin...	一个提供AI工程学习资源和生产级项目的综...	Python, AI, LLM, RAG, Streamlit	9
2026-01-08 18:38:23	ebook2audiobook CPU/GPU Converter from ...	一个将电子书转换为有声书的工具,支持多...	Python, TTS, Docker, Gradio, CUDA	8
2026-01-08 18:38:23	Sponsored by ApyHub: A Modern API Platform t...	一个收集和分类各种免费API的目录项目	API, Web, Development, REST, Open Source	7
2026-01-08 18:38:23	YT PRO Become a Sponsor TipIf you like this pr...	一个增强YouTube观看体验的浏览器扩展,...	JavaScript, Web, Browser Extension, AI, Video	8
2026-01-08 18:38:23	Try our Demo! MiroThinker is MiroMind's Flag...	一个开源搜索代理模型,支持工具增强推理...	Python, AI, Machine Learning, NLP, Search	9

图9 热榜基础数据采集截图

readme 列中包含大量的 Markdown 文本,证明 LONGTEXT 类型设计合理,成功存储了文档。

7.2 AI 增值数据

图10展示ai_summary、ai_tags、ai_score,字段不仅不为空,且内容准确,证明ai_analyze.py脚本集成成功。

ai_summary	ai_tags	ai_score
一个为Claude Code设计的持久内存压缩系统,用于跨会话保存上下文。	JavaScript, Bun, SQLite, Chroma, HTTP	8
Google开发的C++单元测试框架,支持xUnit架构和多种测试功能。	C++, Testing, xUnit, Google, Framework	9
一个提供网站全面开源情报分析的工具,用于安全优化和架构洞察。	Web, Security, Docker, DNS, SSL	8
一个微软开发的Windows系统增强工具集,包含超过25个实用程序。	Windows, C++, .NET, PowerShell, WinGet	8
Google开发的用于序列化结构化数据的数据语言中立、平台中立的机制。	C++, Java, Python, Bazel, Protocol Buffers	9
一个让AI编程助手通过MCP协议控制Chrome浏览器进行自动化调试和性能分...	Node.js, Chrome DevTools, Puppeteer, MCP	8
一个为AI代理提供便携式、单文件内存层的系统,支持快速检索和长期记...	Rust, Python, Node.js, AI, Memory	9
一个提供AI工程学习资源和生产级项目的综合知识库	Python, AI, LLM, RAG, Streamlit	9
一个将电子书转换为有声书的工具,支持多语言和语音克隆。	Python, TTS, Docker, Gradio, CUDA	8
一个收集和分类各种免费API的目录项目	API, Web, Development, REST, Open Source	7
一个增强YouTube观看体验的浏览器扩展,提供视频下载、广告拦截和AI摘...	JavaScript, Web, Browser Extension, AI, Video	8
一个开源搜索代理模型,支持工具增强推理,用于复杂研究任务。	Python, AI, Machine Learning, NLP, Search	9
一个关于Claude提示工程的交互式教程,包含基础到高级的实践练习。	AI, Prompt Engineering, Claude, Google She...	8

图10 热榜 ai_summary、ai_tags、ai_score 数据集成截图

7.3 知识图谱关系数据

展示 repo_url 和 contributor_name 的对应关系,证明 API 采集成功,为图谱构建提供了节点与边的数据。

	id	repo_url	contributor_name	contributor_url	avatar_url	crawl_time
▶	1	https://github.com/thedotmack/claude-mem	thedotmack	https://github.com/thedotmack	https://avatars.githubusercontent.com/u/6839...	2026-01-08 18:41:52
	2	https://github.com/thedotmack/claude-mem	Copilot	https://github.com/apps/copilot-swe-agent	https://avatars.githubusercontent.com/in/1143...	2026-01-08 18:41:52
	3	https://github.com/thedotmack/claude-mem	github-actions[bot]	https://github.com/apps/github-actions	https://avatars.githubusercontent.com/in/1536...	2026-01-08 18:41:52
	4	https://github.com/thedotmack/claude-mem	dreamiurg	https://github.com/dreamiurg	https://avatars.githubusercontent.com/u/3694...	2026-01-08 18:41:52
	5	https://github.com/thedotmack/claude-mem	7Sageer	https://github.com/7Sageer	https://avatars.githubusercontent.com/u/1259...	2026-01-08 18:41:52
	6	https://github.com/thedotmack/claude-mem	delorenj	https://github.com/delorenj	https://avatars.githubusercontent.com/u/2426...	2026-01-08 18:41:52
	7	https://github.com/thedotmack/claude-mem	ToxMox	https://github.com/ToxMox	https://avatars.githubusercontent.com/u/2256...	2026-01-08 18:41:52
	8	https://github.com/thedotmack/claude-mem	kat-bell	https://github.com/kat-bell	https://avatars.githubusercontent.com/u/2244...	2026-01-08 18:41:52
	9	https://github.com/thedotmack/claude-mem	claude	https://github.com/claude	https://avatars.githubusercontent.com/u/8184...	2026-01-08 18:41:52
	10	https://github.com/thedotmack/claude-mem	CrystalDEV	https://github.com/CrystalDEV	https://avatars.githubusercontent.com/u/1657...	2026-01-08 18:41:52
	11	https://github.com/google/googletest	gennadiycivil	https://github.com/gennadiycivil	https://avatars.githubusercontent.com/u/1273...	2026-01-08 18:41:55
	12	https://github.com/google/googletest	dereksmauro	https://github.com/dereksmauro	https://avatars.githubusercontent.com/u/7611...	2026-01-08 18:41:55

图 11 热榜关系数据采集截图

8、数据采集与集成的国内外现状认识

在数据采集与集成领域，国外尤其是欧美国家起步较早，已经建立了极为成熟的技术生态与标准体系。目前，国外的技术重心正从传统的 ETL（提取、转换、加载）模式向 ELT（提取、加载、转换）以及云原生集成转变。随着 Snowflake、Databricks 等云数据平台的普及，国外企业更倾向于将多源异构数据直接采集到云端数据湖中，利用云计算的弹性算力进行高效处理。此外，受 GDPR（通用数据保护条例）等严格法规的影响，国外在采集阶段就高度融合了数据治理理念，特别强调隐私保护（Privacy by Design）、数据血缘追踪以及数据质量的自动化监控，实现了高度的工程化与合规化。

我国在数据采集与集成领域虽然基础软件起步稍晚，但得益于移动互联网和数字经济的爆发式增长，发展速度极快，呈现出鲜明的“应用驱动”特征。依托于头部互联网企业的业务需求，国内在处理海量、高并发的数据采集（如电商大促交易流、短视频用户行为日志）方面积累了世界领先的经验，Apache Flink 等实时计算技术在国内得到了极高水平的应用。同时，随着“数字中国”建设的推进，政府和企业正大力建设跨部门的数据共享交换平台，致力于利用 API 网关和数据中台技术打破长期存在的“数据孤岛”现象。不过，与国外相比，国内在底层通用集成工具的原创性以及数据全生命周期的精细化治理上，仍有提升空间。

通过本次大作业的实践，我深刻认识到自身在数据采集与集成方面仍存在明显的不足。首先是系统架构思维的欠缺，在项目初期，我往往只关注如何写代码把数据“抓”下来，而忽视了数据源的稳定性、反爬策略的复杂性以及采集端的容错机制设计，导致系统在面对异常情况时不够健壮。其次是数据质量意识薄弱，在处理多源数据时，对于非结构化数据的清洗（Data Cleaning）和去重工作做得不够细致，忽略了“垃圾进，垃圾出”的风险。此外，对于数据安全和隐私法律的敏感度也不够，仅停留在技术实现层面，缺乏对合规性的深入思考。

针对上述不足，明确了未来的努力方向。在技术层面，我将不再满足于简单的脚本编写，而是深入学习分布式采集框架（如 Scrapy-Redis）和企业级集成组件（如 Kafka、Canal），提升处理大规模数据的能力，并尝试引入自动化运维（DataOps）的思路来监控采集任务。在认知层面，我将更加注重数据的规范化与标准化，学习国内外先进的数据模型设计。同时，我也会加强对《网络安全法》及数据安全技术的學習，在未来的开发中将安全性和隐私保护作为系统的核心要素，致力于成为一名既懂技术实现又懂数据治理的综合型开发者。

9、心得体会

9.1 课程收获

通过完成这个项目，我在理论和实践层面均有显著提升：首先提升了全栈技术整合能力，我成功将 Python 爬虫（Requests/BS4）、数据库（MySQL/ChromaDB）、人工智能（DeepSeek API）和前端可视化（Streamlit）四大技术栈打通。深刻理解了数据集成不仅是存数据，更是连接各个技术孤岛的桥梁；同时，我还建立了工程化思维，在解决 GitHub API 限流、Emoji 表情存储报错、长文本数据库截断等实际问题的过程中，我学会了正则化处理和事务管理，明白了系统的健壮性

比功能更重要。

另外，在数据采集和集成方面，我完成了 AI 赋能视角的转变，以前认为数据采集就是写正则提取字符串，现在明白可以利用 LLM 进行模糊匹配、摘要生成和情感分析。这种 AI 驱动的开发模式极大地拓宽了我的技术视野。

9.2 存在的不足

尽管项目实现了预期功能，但对照工业级标准，仍存在以下局限：

- **采集效率瓶颈：**目前爬虫采用单线程同步阻塞模式（Sequential），在处理大量页面时效率较低。且尚未实现分布式部署，受限于单机性能。
- **数据实时性不足：**系统依赖手动运行脚本触发更新，缺乏定时任务调度（Scheduler）和增量更新机制，无法做到数据的实时同步。
- **错误恢复机制薄弱：**虽然有简单的异常捕获，但缺乏断点续传功能。如果在采集几千条数据时网络中断，需要从头开始，造成资源浪费。

9.3 改进方向

针对以上不足，未来可以从以下方面进行优化：

- **引入分布式架构：**使用 Scrapy-Redis 框架重构爬虫模块，实现多节点分布式采集，大幅提升爬取速度。
- **自动化运维：**引入 Apache Airflow 或简单的 Crontab 定时任务，实现每日凌晨自动执行采集与 LLM 分析脚本，实现全自动化运行。
- **深度微调模型：**目前仅使用了 Prompt Engineering 调用通用大模型。未来可以尝试收集高质量的技术文档数据，对开源小模型（如 Qwen 或 Llama 3）进行 LoRA 微调，使其更专注于代码领域的理解，降低 API 调用成本并提升分析精准度。

参考文献

- [1] Lewis P, Perez E, Piktus A, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks[J]. Advances in neural information processing systems, 2020, 33: 9459-9474.
- [2] Reimers N, Gurevych I. Sentence-bert: Sentence embeddings using siamese bert-networks[J]. arXiv preprint arXiv:1908.10084, 2019.
- [3] Zhao W X, Zhou K, Li J, et al. A survey of large language models[J]. arXiv preprint arXiv:2303.18223, 2023, 1(2).
- [4] Khan B, Jan S, Khan W, et al. An Overview of ETL Techniques, Tools, Processes and Evaluations in Data Warehousing[J]. Journal on Big Data, 2024, 6.
- [5] Kobourov S G. Spring embedders and force directed graph drawing algorithms[J]. arXiv preprint arXiv:1201.3011, 2012.

附录

小组分工：

：数据采集、平台设计构想、平台搭建、Deepseek 引入、报告撰写。

吉稼熠：数据采集、平台设计构想、知识图谱构建、PPT 制作。

create.sql

```
CREATE DATABASE gitTrend_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE gitTrend_db;
CREATE TABLE `github_trending_day` (
  `id` int NOT NULL AUTO_INCREMENT,
  `ranking` int NOT NULL COMMENT '榜单排名',
  `repo` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL COMMENT '仓库名称, 如 owner/repo',
  `url` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL COMMENT '仓库链接',
  `description` text COLLATE utf8mb4_unicode_ci COMMENT '仓库描述',
  `lang` varchar(50) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '主要编程语言',
  `stars` varchar(50) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '总 Star 数',
  `forks` varchar(50) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '总 Fork 数',
  `added_stars` varchar(50) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '今日新增 Star',
  `time` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '爬取时间',
  `readme` longtext COLLATE utf8mb4_unicode_ci COMMENT 'README 原始内容',
  `ai_summary` varchar(500) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT 'AI 生成的摘要',
  `ai_tags` varchar(255) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT 'AI 生成的标签',
  `ai_score` int DEFAULT NULL COMMENT 'AI 评分',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `repo_contributors` (
  `id` int NOT NULL AUTO_INCREMENT,
  `repo_url` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL COMMENT '关联的仓库链接',
  `contributor_name` varchar(100) COLLATE utf8mb4_unicode_ci NOT NULL COMMENT '贡献者用户名',
  `contributor_url` varchar(255) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '贡献者主页链接',
  `avatar_url` varchar(255) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '头像链接',
  `crawl_time` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '爬取时间',
  PRIMARY KEY (`id`),
  KEY `idx_repo` (`repo_url`),
  KEY `idx_contributor` (`contributor_name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```